

DOCUMENTO TÉCNICO

Modelo de datos v1.0 — Firestore

Modelo de datos

Plataforma SaaS de gestión de turnos para transporte urbano

Autor: Alberto

Mayo 2026

Documento complementario a la especificación funcional

Índice de contenidos

1. Introducción
2. Principios de diseño
3. Estrategia multi-tenant
4. Mapa general de colecciones
5. Detalle de las colecciones
 - 5.1. tenants
 - 5.2. centros
 - 5.3. users
 - 5.4. trabajadores
 - 5.5. lineas
 - 5.6. paradas
 - 5.7. frecuencias
 - 5.8. frecuencias_excepcionales
 - 5.9. tipos_turno
 - 5.10. servicios
 - 5.11. cuadrantes
 - 5.12. versiones_cuadrante
 - 5.13. asignaciones
 - 5.14. cambios_asignaciones
 - 5.15. preferencias_permanentes
 - 5.16. preferencias_puntuales
 - 5.17. solicitudes_intercambio
 - 5.18. incidencias
 - 5.19. calendario_festivos
 - 5.20. configuracion_convenio
 - 5.21. notificaciones
6. Diagrama de relaciones entre entidades
7. Reglas de seguridad de Firestore
8. Estrategia de índices
9. Próximos pasos

1. Introducción

Este documento describe el modelo de datos completo de la plataforma SaaS de gestión de turnos para empresas de transporte urbano. Recoge todas las colecciones de Firestore que componen el sistema, los campos de cada documento, sus tipos, las relaciones entre entidades, las reglas de seguridad multi-tenant y la estrategia de índices.

Este modelo es la base sobre la que se construirá toda la lógica de la aplicación. Cualquier cambio posterior en las decisiones aquí tomadas tendrá impacto en el resto del desarrollo, por lo que se ha diseñado pensando en la escalabilidad, la integridad de los datos y el aislamiento entre clientes.

El documento está pensado tanto para servir de referencia técnica durante el desarrollo como para ser revisado por el autor antes de iniciar la fase de codificación.

2. Principios de diseño

El modelo se rige por los siguientes principios fundamentales, derivados de las decisiones tomadas durante la fase de definición:

Aislamiento total entre tenants

Cada empresa cliente opera en un entorno completamente aislado. Bajo ninguna circunstancia un usuario de una empresa puede acceder a datos de otra. Este aislamiento se garantiza mediante el campo `tenantId` presente en todos los documentos y mediante reglas de seguridad estrictas.

Granularidad por documento

Cada entidad significativa es un documento independiente. Especialmente las asignaciones del cuadrante: una por cada combinación conductor + día. Esto permite modificaciones puntuales sin afectar al resto, soporte de tiempo real y trazabilidad de cambios.

Materialización selectiva

Los servicios diarios no se materializan masivamente. Solo se crea el documento del servicio cuando hay algo que registrar sobre él (asignación, incidencia, modificación). El resto se calcula al vuelo a partir de las frecuencias configuradas.

Trazabilidad estilo Git

Todo cambio sobre un cuadrante publicado queda registrado. Las versiones publicadas se almacenan como snapshots ligeros y los cambios entre versiones se guardan como diffs. El histórico es consultable e inmutable.

Configurabilidad por tenant

Las reglas operativas y legales (convenio, plazos, parámetros del optimizador) son configurables por cliente. El producto es uno solo, pero se adapta a la realidad de cada empresa.

3. Estrategia multi-tenant

La plataforma adopta el modelo de colecciones planas (flat collections) con identificador de tenant en cada documento. Es decir, todas las colecciones son colecciones de primer nivel en Firestore y cada documento contiene un campo `tenantId` que indica a qué empresa pertenece.

Justificación frente a la alternativa de subcolecciones

La alternativa habitual sería anidar todas las colecciones bajo `/tenants/{tenantId}/....` Tras evaluar ambas opciones, se opta por colecciones planas por las siguientes razones:

- Permite consultas globales para el super administrador sin necesidad de iterar por todos los tenants.
- Simplifica los índices compuestos y las reglas de seguridad.
- Facilita la portabilidad de datos y los procesos de exportación.
- Es el patrón recomendado por Google para SaaS multi-tenant a partir de cierto tamaño.

Campos comunes en todos los documentos

Toda colección de la aplicación incluye, además de sus campos específicos, los siguientes campos transversales:

Campo	Tipo	Descripción
<code>tenantId</code>	<code>string</code>	Identificador de la empresa cliente. Filtra acceso multi-tenant.
<code>centroId</code>	<code>string?</code>	Identificador del centro operativo (opcional, según colección).
<code>createdAt</code>	<code>Timestamp</code>	Fecha y hora de creación del documento.
<code>createdBy</code>	<code>string</code>	ID del usuario que creó el documento.
<code>updatedAt</code>	<code>Timestamp</code>	Fecha y hora de la última modificación.
<code>updatedBy</code>	<code>string</code>	ID del usuario que realizó la última modificación.

Por concisión, en las descripciones de cada colección que vienen a continuación no se repetirán estos campos comunes, salvo cuando aporten alguna particularidad relevante.

4. Mapa general de colecciones

El sistema se compone de las siguientes colecciones, agrupadas por dominio funcional:

Estructura organizativa

- tenants — empresas cliente.
- centros — centros operativos dentro de cada empresa.
- users — usuarios de la plataforma.
- trabajadores — conductores y otros trabajadores.

Operativa de líneas y servicios

- líneas — líneas de transporte.
- paradas — paradas reutilizables entre líneas.
- frecuencias — frecuencias estándar por línea, tramo y tipo de día.
- frecuencias_excepcionales — sustituciones puntuales para fechas concretas.
- tipos_turno — plantillas de turno definidas por la empresa.
- servicios — servicios materializados (solo los que tienen asignación o incidencia).

Cuadrante y asignaciones

- cuadrantes — metadatos del cuadrante (mes, centro, estado, versión actual).
- versiones_cuadrante — snapshots de versiones publicadas.
- asignaciones — asignaciones individuales conductor + día + turno.
- cambios_asignaciones — log de cambios entre versiones.

Preferencias e intercambios

- preferencias_permanentes — perfil de preferencias permanentes del conductor.
- preferencias_puntuales — días concretos prioritarios o solicitudes puntuales.
- solicitudes_intercambio — peticiones de intercambio entre conductores.

Calendario y configuración

- incidencias — bajas, ausencias y eventos operativos.
- calendario_festivos — festivos por tenant.
- configuracion_convenio — reglas legales por tenant.

Comunicación y auditoría

- notificaciones — notificaciones enviadas a usuarios.

5. Detalle de las colecciones

A continuación se describe en profundidad cada colección. Para cada una se indica su propósito, los campos que componen sus documentos, un ejemplo concreto y los índices recomendados.

5.1. tenants

Propósito

Empresa cliente de la plataforma. Cada tenant es una organización que ha contratado el servicio y dispone de su propio espacio aislado dentro del sistema.

Campos

Campo	Tipo	Descripción
id	string	Identificador único del tenant (ej: 'alsa', 'tcc', 'auvasa').
nombre	string	Nombre comercial de la empresa.
razonSocial	string	Razón social legal.
cif	string	CIF o NIF de la empresa.
plan	string	Plan contratado: 'basico' 'pro' 'enterprise'.
estado	string	'activo' 'suspendido' 'cancelado'.
fechaAlta	Timestamp	Fecha en que se activó el contrato.
fechaProximaRenovacion	Timestamp	Fecha de la próxima renovación o facturación.
contactoPrincipal	object	Datos del contacto principal: nombre, email, teléfono.
zonaHoraria	string	Zona horaria operativa (ej: 'Europe/Madrid').
comunidadAutonoma	string	Comunidad autónoma para precarga de festivos.
provincia	string	Provincia para precarga de festivos.

Ejemplo

```
{
  id: "alsa",
  nombre: "ALSA",
  razonSocial: "ALSA Grupo S.L.U.",
  cif: "B33112345",
  plan: "pro",
  estado: "activo",
  fechaAlta: "2026-05-15T10:00:00Z",
  contactoPrincipal: {
    nombre: "Juan Pérez",
    email: "juan.perez@alsa.es",
    telefono: "+34 600 123 456"
  },
  zonaHoraria: "Europe/Madrid",
  comunidadAutonoma: "Asturias",
}
```



```

    provincia: "Asturias"
  }

```

Índices recomendados

- estado (para filtrar tenants activos).
- plan (para análisis comerciales).

5.2. centros

Propósito

Centro operativo dentro de un tenant. Una empresa puede tener uno o varios centros (por ejemplo, una empresa que opera el urbano de varias ciudades). Cada centro gestiona su propia plantilla, sus líneas y su cuadrante de manera independiente.

Campos

Campo	Tipo	Descripción
id	string	Identificador único del centro.
tenantId	string	Tenant al que pertenece.
nombre	string	Nombre del centro (ej: 'Murcia capital', 'Cartagena').
direccion	string	Dirección física de las cocheras.
coordenadas	GeoPoint?	Coordenadas geográficas (opcional, para V2).
jefeTraficoIds	string[]	IDs de los usuarios con rol jefe de tráfico de este centro.
activo	boolean	Centro operativo o desactivado.

Ejemplo

```

{
  id: "alsa_murcia",
  tenantId: "alsa",
  nombre: "Murcia Capital",
  direccion: "Polígono Industrial Oeste, 30169 San Ginés",
  jefeTraficoIds: ["user_juan_jefe"],
  activo: true
}

```

Índices recomendados

- tenantId + activo.

5.3. users

Propósito

Usuarios de la plataforma con capacidad de acceder a la aplicación. Vinculado a un usuario de Firebase Auth a través del UID. La distinción de rol determina sus permisos.

Campos

Campo	Tipo	Descripción
id	string	UID del usuario en Firebase Auth.
tenantId	string?	Tenant al que pertenece (null para super admin).
centroId	string?	Centro asignado (solo para jefes de tráfico).
email	string	Email del usuario.
nombre	string	Nombre completo.
telefono	string?	Teléfono de contacto.
rol	string	'super_admin' 'jefe_trafico' 'conductor'.
trabajadorId	string?	Si es conductor, ID en la colección trabajadores.
activo	boolean	Si puede iniciar sesión actualmente.
ultimoAcceso	Timestamp?	Última vez que el usuario inició sesión.
preferenciasUI	object?	Configuración de la interfaz: idioma, notificaciones, etc.

Notas importantes sobre seguridad

El tenantId, centroId y rol también se almacenan como custom claims en el token de Firebase Auth. Esto permite que las reglas de seguridad de Firestore filtren accesos sin necesidad de leer el documento del usuario en cada operación.

Ejemplo

```
{
  id: "uid_firebase_auth_pedro",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  email: "pedro.gomez@alsa.es",
  nombre: "Pedro Gómez Martínez",
  telefono: "+34 600 111 222",
  rol: "conductor",
  trabajadorId: "trab_pedro_gomez",
  activo: true,
  ultimoAcceso: "2026-05-04T07:30:00Z"
}
```

Índices recomendados

- tenantId + rol.
- tenantId + centroId.
- trabajadorId.

5.4. trabajadores

Propósito

Trabajadores de la empresa cliente. En el MVP solo se gestionan conductores, pero el modelo está preparado para acoger otras categorías en versiones futuras.

Campos

Campo	Tipo	Descripción
id	string	Identificador único del trabajador.
tenantId	string	Tenant al que pertenece.
centroId	string	Centro al que pertenece.
userId	string?	ID del user vinculado (puede no tener cuenta de acceso).
categoria	string	'conductor' (en MVP, único valor admitido).
nombre	string	Nombre completo.
dni	string	DNI o documento identificativo.
telefono	string	Teléfono de contacto.
email	string	Email de contacto.
fechaAlta	Timestamp	Fecha de incorporación a la empresa.
fechaAntigüedad	Timestamp	Fecha de antigüedad reconocida (puede diferir del alta).
fechaBaja	Timestamp?	Fecha de baja en la empresa.
estado	string	'activo' 'baja_temporal' 'baja_definitiva' 'vacaciones'.
lineasPreferentes	string[]	IDs de líneas que el conductor domina y suele cubrir.
lineasSecundarias	string[]	IDs de líneas que conoce y puede cubrir si es necesario.
tiposTurnoPermitidos	string[]	IDs de tipos de turno que este conductor puede hacer.
tiposTurnoExcluidos	string[]	IDs de tipos de turno que NO puede o no quiere hacer.
maxHorasSemanales	number?	Límite individual si difiere del estándar del convenio.
prioridadVacaciones	number	Posición de prioridad para elección de vacaciones (menor número = más prioridad).
observaciones	string?	Notas internas del jefe de tráfico.

Ejemplo

```
{
  id: "trab_pedro_gomez",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  userId: "uid_firebase_auth_pedro",
  categoria: "conductor",
  nombre: "Pedro Gómez Martínez",
  dni: "12345678X",
  telefono: "+34 600 111 222",
  email: "pedro.gomez@alsa.es",
  fechaAlta: "2015-03-12T00:00:00Z",
  fechaAntigüedad: "2015-03-12T00:00:00Z",
  estado: "activo",
  lineasPreferentes: ["linea_5", "linea_7"],
  lineasSecundarias: ["linea_12", "linea_3"],
  tiposTurnoPermitidos: ["mañana_corto", "mañana_largo", "tarde_corto", "tarde_largo"],
}
```

```

tiposTurnoExcluidos: ["nocturno"],
prioridadVacaciones: 12,
observaciones: ""
}

```

Índices recomendados

- tenantId + centroId + estado.
- tenantId + categoria + estado.
- userId.

5.5. lineas

Propósito

Línea de transporte operada por la empresa, con su recorrido (paradas), color identificativo y demás atributos.

Campos

Campo	Tipo	Descripción
id	string	Identificador único de la línea.
tenantId	string	Tenant al que pertenece.
centroId	string	Centro que opera la línea.
codigo	string	Código corto de la línea (ej: 'L5', '7B', 'N1').
nombre	string	Nombre descriptivo (ej: 'Centro - Universidad').
color	string	Color en formato hexadecimal (ej: '#FF5733').
tipo	string	'urbana' 'nocturna' 'refuerzo' 'especial'.
paradasIds	string[]	IDs de las paradas en orden de recorrido.
activa	boolean	Si la línea está operativa.
observaciones	string?	Notas operativas relevantes.

Ejemplo

```

{
  id: "linea_5",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  codigo: "L5",
  nombre: "Centro - Universidad",
  color: "#1F77B4",
  tipo: "urbana",
  paradasIds: ["parada_plaza_mayor", "parada_glorieta", "parada_facultad",
    "parada_residencia", "parada_universidad"],
  activa: true
}

```

Índices recomendados

- tenantId + centroId + activa.

- tenantId + tipo.

5.6. paradas

Propósito

Parada física. Las paradas son entidades reutilizables entre líneas, de modo que una parada como Plaza Mayor pueda ser servida por la línea 5 y la línea 7 simultáneamente.

Campos

Campo	Tipo	Descripción
id	string	Identificador único de la parada.
tenantId	string	Tenant al que pertenece.
centroId	string	Centro al que pertenece.
codigo	string	Código corto (ej: 'P001').
nombre	string	Nombre descriptivo (ej: 'Plaza Mayor').
direccion	string?	Dirección textual.
coordenadas	GeoPoint?	Coordenadas geográficas para visualización en mapa (V2).
lineasIds	string[]	IDs de las líneas que pasan por esta parada (denormalizado para búsquedas rápidas).
activa	boolean	Si la parada está operativa.

Nota sobre denormalización

El campo lineasIds es redundante con la información de la colección líneas (que ya almacena paradasIds). Esta denormalización es deliberada: permite responder rápidamente a la pregunta '¿qué líneas pasan por esta parada?' sin necesidad de consultar todas las líneas. La consistencia se mantiene mediante una Cloud Function que actualiza ambos lados al modificar el recorrido de una línea.

Ejemplo

```
{
  id: "parada_plaza_mayor",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  codigo: "P001",
  nombre: "Plaza Mayor",
  direccion: "Plaza del Cardenal Belluga s/n",
  lineasIds: ["linea_5", "linea_7", "linea_12"],
  activa: true
}
```

Índices recomendados

- tenantId + centroId + activa.
- lineasIds (array-contains).

5.7. frecuencias

Propósito

Frecuencias estándar de paso de cada línea por tramo horario y tipo de día. Estas frecuencias son las que aplican habitualmente y se usan para calcular los servicios de cualquier día sin necesidad de materializarlos.

Campos

Campo	Tipo	Descripción
id	string	Identificador único de la frecuencia.
tenantId	string	Tenant al que pertenece.
centroId	string	Centro al que pertenece.
lineaId	string	Línea a la que aplica.
tipoDia	string	'laborable' 'sabado' 'domingo' 'festivo'.
horaInicio	string	Hora de inicio del tramo en formato HH:mm.
horaFin	string	Hora de fin del tramo en formato HH:mm.
intervaloMinutos	number	Cada cuántos minutos sale una expedición.
sentido	string	'ida' 'vuelta' 'circular'.
activa	boolean	Si la frecuencia está vigente.

Ejemplo

```
{
  id: "frec_l5_lab_06_10",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  lineaId: "linea_5",
  tipoDia: "laborable",
  horaInicio: "06:00",
  horaFin: "10:00",
  intervaloMinutos: 12,
  sentido: "ida",
  activa: true
}
```

Índices recomendados

- tenantId + lineaId + tipoDia + activa.

5.8. frecuencias_excepcionales

Propósito

Frecuencias que reemplazan a las habituales en fechas concretas. Útiles para eventos puntuales (conciertos, partidos, manifestaciones) en los que la empresa altera el servicio de manera planificada.

Comportamiento

Las frecuencias excepcionales reemplazan completamente a las habituales del tramo y línea afectados durante la fecha indicada. Es decir, durante una franja excepcional, las expediciones programadas son las definidas en la excepción y NO las habituales.

Campos

Campo	Tipo	Descripción
id	string	Identificador único.
tenantId	string	Tenant.
centroId	string	Centro.
lineaId	string	Línea afectada.
fecha	string	Fecha en formato YYYY-MM-DD.
horaInicio	string	Inicio del tramo excepcional.
horaFin	string	Fin del tramo excepcional.
intervaloMinutos	number	Intervalo durante la excepción.
motivo	string	Descripción del motivo (concierto, evento, etc.).
creadaPor	string	ID del jefe de tráfico que la creó.

Ejemplo

```
{
  id: "frec_exc_l5_2026_06_15",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  lineaId: "linea_5",
  fecha: "2026-06-15",
  horaInicio: "20:00",
  horaFin: "23:30",
  intervaloMinutos: 6,
  motivo: "Concierto en estadio - refuerzo de servicio",
  creadaPor: "user_juan_jefe"
}
```

Índices recomendados

- tenantId + centroId + fecha.
- tenantId + lineaId + fecha.

5.9. tipos_turno

Propósito

Plantillas de tipo de turno definidas por la empresa. Cada empresa configura sus propios tipos según su operativa habitual.

Campos

Campo	Tipo	Descripción
id	string	Identificador único del tipo de turno.

Campo	Tipo	Descripción
tenantId	string	Tenant al que pertenece.
centroId	string	Centro (los tipos pueden variar por centro).
codigo	string	Código corto (ej: 'M-C', 'T-L').
nombre	string	Nombre descriptivo (ej: 'Mañana corto').
horaInicio	string	Hora de inicio del turno (HH:mm).
horaFin	string	Hora de fin del turno (HH:mm).
duracionMinutos	number	Duración total en minutos.
duracionConduccionMinutos	number	Tiempo efectivo de conducción.
esPartido	boolean	Si el turno es jornada partida.
bloques	object[]?	Si es partido, los bloques que lo componen: [{horaInicio, horaFin}].
esNocturno	boolean	Si se considera nocturno a efectos de convenio.
activo	boolean	Si el tipo de turno está vigente.
color	string?	Color para visualización en el cuadrante.

Ejemplo de turno simple

```
{
  id: "turno_mañana_largo",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  codigo: "M-L",
  nombre: "Mañana largo",
  horaInicio: "06:00",
  horaFin: "14:00",
  duracionMinutos: 480,
  duracionConduccionMinutos: 420,
  esPartido: false,
  esNocturno: false,
  activo: true,
  color: "#FFD700"
}
```

Ejemplo de turno partido

```
{
  id: "turno_partido_1",
  codigo: "P-1",
  nombre: "Partido mañana-tarde",
  horaInicio: "06:00",
  horaFin: "21:00",
  duracionMinutos: 480,
  esPartido: true,
  bloques: [
    { horaInicio: "06:00", horaFin: "10:00" },
    { horaInicio: "17:00", horaFin: "21:00" }
  ],
  esNocturno: false,
```



```
    activo: true
  }
```

Índices recomendados

- tenantId + centroId + activo.

5.10. servicios

Propósito

Servicios materializados (expediciones concretas) que tienen alguna asignación o anotación. Solo se crea el documento cuando hay algo que registrar; el resto de servicios del día se calculan al vuelo a partir de las frecuencias.

Campos

Campo	Tipo	Descripción
id	string	Identificador único del servicio.
tenantId	string	Tenant.
centroId	string	Centro.
lineaId	string	Línea.
fecha	string	Fecha en formato YYYY-MM-DD.
horaSalida	string	Hora de salida del servicio (HH:mm).
horaFin	string	Hora de fin estimada del servicio.
sentido	string	'ida' 'vuelta' 'circular'.
origenParadaId	string	Parada de origen.
destinoParadaId	string	Parada de destino.
estado	string	'planificado' 'en_curso' 'completado' 'cancelado' 'incidencia'.
asignacionIds	string[]	IDs de las asignaciones que cubren este servicio (uno o varios para relevos).
incidenciaId	string?	Si hubo incidencia asociada.
observaciones	string?	Notas operativas.

Ejemplo

```
{
  id: "srv_2026_05_05_l5_0612",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  lineaId: "linea_5",
  fecha: "2026-05-05",
  horaSalida: "06:12",
  horaFin: "06:48",
  sentido: "ida",
  origenParadaId: "parada_plaza_mayor",
  destinoParadaId: "parada_universidad",
```

```

    estado: "planificado",
    asignacionIds: ["asg_pedro_2026_05_05"]
  }

```

Índices recomendados

- tenantId + centroId + fecha + estado.
- tenantId + linealId + fecha.

5.11. cuadrantes

Propósito

Metadatos del cuadrante mensual de un centro. Es un documento ligero que describe el estado y la versión actual de un cuadrante, sin contener las asignaciones (que viven en su propia colección).

Campos

Campo	Tipo	Descripción
id	string	Identificador único (ej: 'cuadr_alsa_murcia_2026_05').
tenantId	string	Tenant.
centroId	string	Centro.
año	number	Año del cuadrante.
mes	number	Mes del cuadrante (1-12).
estado	string	'borrador' 'publicado' 'archivado'.
versionActual	number	Número de la versión actual.
fechaPublicacion	Timestamp?	Fecha de publicación inicial.
ultimaModificacion	Timestamp	Fecha de la última modificación.
generadoPor	string	'optimizador' 'manual'.
estadisticas	object	Métricas resumidas: n° asignaciones, satisfacción media, alertas, etc.
alertasConvenio	object[]	Alertas detectadas durante la generación.

Ejemplo

```

{
  id: "cuadr_alsa_murcia_2026_05",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  año: 2026,
  mes: 5,
  estado: "publicado",
  versionActual: 3,
  fechaPublicacion: "2026-04-25T18:00:00Z",
  ultimaModificacion: "2026-05-02T11:40:00Z",
  generadoPor: "optimizador",
  estadisticas: {
    totalAsignaciones: 2700,
  }
}

```

```

    conductoresAsignados: 90,
    satisfaccionMedia: 0.87,
    preferenciasNoSatisfechas: 14
  },
  alertasConvenio: []
}

```

Índices recomendados

- tenantId + centroId + año + mes.
- tenantId + estado.

5.12. versiones_cuadrante

Propósito

Snapshots de cada versión publicada del cuadrante. Permite reconstruir cómo estaba el cuadrante en un momento dado y comparar entre versiones (estilo Git).

Campos

Campo	Tipo	Descripción
id	string	ID de la versión.
tenantId	string	Tenant.
cuadranteId	string	Cuadrante al que pertenece.
numeroVersion	number	Número secuencial de versión (1, 2, 3...).
fecha	Timestamp	Fecha de creación de la versión.
autorId	string	Usuario que generó la versión.
motivo	string	Motivo o descripción del cambio.
snapshotAsignaciones	object[]	Snapshot completo de las asignaciones en este momento.
cambiosRespectoAnterior	string[]	IDs de los documentos en cambios_asignaciones que componen este versionado.

Nota sobre el snapshot

El snapshot puede ser una lista compacta {trabajadorId, fecha, tipoTurnoId} en lugar del documento completo de cada asignación, para reducir el tamaño. Ante consultas detalladas, se reconstruye con los IDs.

Ejemplo

```

{
  id: "ver_cuadr_alsa_murcia_2026_05_v3",
  tenantId: "alsa",
  cuadranteId: "cuadr_alsa_murcia_2026_05",
  numeroVersion: 3,
  fecha: "2026-05-02T11:40:00Z",
  autorId: "user_juan_jefe",
  motivo: "Intercambio aprobado entre Pedro y Juan (8 mayo)",
  snapshotAsignaciones: [

```

```

    { trabajadorId: "trab_pedro", fecha: "2026-05-08", tipoTurnoId:
"turno_mañana_largo" },
    { trabajadorId: "trab_juan", fecha: "2026-05-08", tipoTurnoId:
"turno_tarde_corto" }
    // ... resto del cuadrante
  ],
  cambiosRespectoAnterior: ["camb_xyz123", "camb_abc456"]
}

```

Índices recomendados

- tenantId + cuadranteId + numeroVersion.

5.13. asignaciones

Propósito

Asignación individual de un conductor a un turno en un día concreto. Es el documento granular del cuadrante: existe uno por cada combinación conductor + día con turno asignado. Soporta turnos simples, múltiples servicios y relevos mediante el array de tramos.

Campos

Campo	Tipo	Descripción
id	string	Identificador único.
tenantId	string	Tenant.
centroId	string	Centro.
cuadranteId	string	Cuadrante al que pertenece.
trabajadorId	string	Conductor asignado.
fecha	string	Fecha en formato YYYY-MM-DD.
tipoTurnoId	string	Tipo de turno asignado.
horaInicio	string	Hora de inicio efectiva (puede coincidir o no con la del tipo de turno).
horaFin	string	Hora de fin efectiva.
duracionMinutos	number	Duración total.
esPersonalizado	boolean	True si es un turno fuera de plantilla.
tramosServicio	object[]	Tramos de servicio cubiertos durante el turno.
esReserva	boolean	Si la asignación es como conductor de reserva.
tipoReserva	string?	'presencial' 'localizable' (si esReserva=true).
estadoAsignacion	string	'planificada' 'realizada' 'modificada' 'cancelada'.
observaciones	string?	Notas del jefe de tráfico.

Estructura del campo tramosServicio

Cada tramo describe un periodo dentro del turno en el que el conductor cubre un servicio concreto, posiblemente compartido con otros conductores en relevo.

```

tramosServicio: [
  {
    servicioId: "srv_2026_05_05_l5_0612",
    lineaId: "linea_5",
    horaInicio: "06:12",
    horaFin: "10:00",
    paradaInicio: "parada_plaza_mayor", // donde recoge el bus
    paradaFin: "parada_universidad",    // donde lo deja (o entrega en
relevo)
    esRelevo: false                      // si el tramo termina con relevo en
línea
  },
  {
    servicioId: "srv_2026_05_05_l7_1100",
    lineaId: "linea_7",
    horaInicio: "11:00",
    horaFin: "14:00",
    paradaInicio: "parada_centro",
    paradaFin: "parada_hospital",
    esRelevo: false
  }
]

```

Ejemplo de asignación simple

```

{
  id: "asg_pedro_2026_05_05",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  cuadranteId: "cuadr_alsa_murcia_2026_05",
  trabajadorId: "trab_pedro_gomez",
  fecha: "2026-05-05",
  tipoTurnoId: "turno_mañana_largo",
  horaInicio: "06:00",
  horaFin: "14:00",
  duracionMinutos: 480,
  esPersonalizado: false,
  tramosServicio: [
    {
      servicioId: "srv_2026_05_05_l5_continuo",
      lineaId: "linea_5",
      horaInicio: "06:00",
      horaFin: "14:00",
      esRelevo: false
    }
  ],
  esReserva: false,
  estadoAsignacion: "planificada"
}

```

Índices recomendados

- tenantId + centroId + fecha.
- tenantId + trabajadorId + fecha.
- cuadranteId + fecha.

- tenantId + esReserva + fecha.

5.14. cambios_asignaciones

Propósito

Log inmutable de los cambios realizados sobre asignaciones de cuadrantes publicados. Cada cambio queda registrado con el valor anterior, el nuevo, el motivo y el usuario que lo realizó. Es el equivalente a los commits de un repositorio Git.

Campos

Campo	Tipo	Descripción
id	string	Identificador único del cambio.
tenantId	string	Tenant.
asignacionId	string	Asignación afectada.
cuadranteId	string	Cuadrante.
versionAnterior	number	Número de versión antes del cambio.
versionNueva	number	Número de versión tras el cambio.
tipoCambio	string	'creacion' 'modificacion' 'cancelacion' 'intercambio'.
fecha	Timestamp	Cuándo se hizo el cambio.
autorId	string	Usuario que realizó el cambio.
motivo	string	Texto descriptivo.
valorAnterior	object?	Snapshot del documento antes del cambio (null si es creación).
valorNuevo	object?	Snapshot del documento tras el cambio (null si es cancelación).
solicitudIntercambioId	string?	Si el cambio proviene de un intercambio.
incidenciaId	string?	Si el cambio proviene de una incidencia.

Ejemplo

```
{
  id: "camb_xyz123",
  tenantId: "alsa",
  asignacionId: "asg_pedro_2026_05_08",
  cuadranteId: "cuadr_alsa_murcia_2026_05",
  versionAnterior: 2,
  versionNueva: 3,
  tipoCambio: "intercambio",
  fecha: "2026-05-02T11:40:00Z",
  autorId: "user_juan_jefe",
  motivo: "Intercambio aprobado entre Pedro y Juan",
  valorAnterior: { tipoTurnoId: "turno_tarde_corto", horaInicio: "14:00",
horaFin: "20:00" },
  valorNuevo: { tipoTurnoId: "turno_mañana_largo", horaInicio: "06:00",
horaFin: "14:00" },
}
```

```
solicitudIntercambioId: "sol_int_2026_05_02_001"
}
```

Índices recomendados

- tenantId + asignacionId + fecha.
- tenantId + cuadrantId + versionNueva.

5.15. preferencias_permanentes

Propósito

Perfil de preferencias permanentes de cada conductor. Configurado una vez y aplicado por defecto. Existe un único documento por conductor.

Campos

Campo	Tipo	Descripción
id	string	Coincide con trabajadorId.
tenantId	string	Tenant.
trabajadorId	string	Conductor al que pertenecen.
turnoPreferido	string	'mañana' 'tarde' 'indiferente'.
tipoDescansoPreferido	string	'alterno' 'consecutivo' 'indiferente'.
descansoTrasTardeMañana	string	'preferir_descanso' 'indiferente'.
preferenciaReserva	string	'evitar' 'indiferente' 'preferir'.
findeSemanaPreferidosLibres	boolean	Si prefiere fines de semana libres en general.
ultimaActualizacion	Timestamp	Cuándo se modificaron por última vez.

Ejemplo

```
{
  id: "pref_perm_pedro",
  tenantId: "alsa",
  trabajadorId: "trab_pedro_gomez",
  turnoPreferido: "mañana",
  tipoDescansoPreferido: "consecutivo",
  descansoTrasTardeMañana: "preferir_descanso",
  preferenciaReserva: "indiferente",
  finesSemanaPreferidosLibres: true,
  ultimaActualizacion: "2026-03-15T18:30:00Z"
}
```

Índices recomendados

- tenantId + trabajadorId.

5.16. preferencias_puntuales

Propósito

Preferencias puntuales del conductor para fechas concretas, registrables hasta con 12 meses de antelación. Cada documento representa una solicitud para una fecha específica.

Campos

Campo	Tipo	Descripción
id	string	Identificador único.
tenantId	string	Tenant.
trabajadorId	string	Conductor.
fecha	string	Fecha solicitada YYYY-MM-DD.
tipo	string	'librar' 'turno_especifico' 'evitar_turno'.
valorEspecifico	string?	Para 'turno_especifico' o 'evitar_turno', el ID del tipo de turno.
prioridad	number	Nivel de prioridad para el optimizador (1-5, donde 5 es máxima).
motivo	string?	Motivo libre indicado por el conductor.
estado	string	'pendiente' 'aplicada' 'no_aplicada'.
explicacionNoAplicada	string?	Si no se pudo aplicar, motivo del optimizador.
fechaCreacion	Timestamp	Cuándo se solicitó.

Ejemplo

```
{
  id: "pref_punt_pedro_2026_10_04",
  tenantId: "alsa",
  trabajadorId: "trab_pedro_gomez",
  fecha: "2026-10-04",
  tipo: "librar",
  prioridad: 5,
  motivo: "Cumpleaños de mi hija",
  estado: "pendiente",
  fechaCreacion: "2026-05-01T10:00:00Z"
}
```

Índices recomendados

- tenantId + trabajadorId + fecha.
- tenantId + fecha + estado.

5.17. solicitudes_intercambio

Propósito

Solicitudes de intercambio entre conductores. Solo se permiten intercambios sobre fechas que ya están en cuadrantes publicados (horizonte de 2 meses).

Campos

Campo	Tipo	Descripción
id	string	Identificador único.
tenantId	string	Tenant.
centroId	string	Centro.
solicitanteId	string	Trabajador que inicia el intercambio.
receptorId	string?	Trabajador objetivo (puede ser null si es oferta abierta).
asignacionOrigenId	string	Asignación que cede el solicitante.
asignacionDestinoId	string	Asignación que recibe el solicitante (la del receptor).
estado	string	'pendiente_receptor' 'pendiente_jefe' 'aprobada' 'rechazada' 'expirada' 'cancelada'.
fechaSolicitud	Timestamp	Cuándo se inició la solicitud.
fechaRespuestaReceptor	Timestamp?	Cuándo respondió el receptor.
fechaResolucion	Timestamp?	Cuándo resolvió el jefe de tráfico.
resueltaPor	string?	ID del jefe de tráfico que resolvió.
motivoRechazo	string?	Si se rechazó, motivo.
validacionesAutomaticas	object	Resultado de validaciones automáticas: descansos, horas, líneas.

Ejemplo

```
{
  id: "sol_int_2026_05_02_001",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  solicitanteId: "trab_pedro_gomez",
  receptorId: "trab_juan_lopez",
  asignacionOrigenId: "asg_pedro_2026_05_08",
  asignacionDestinoId: "asg_juan_2026_05_08",
  estado: "aprobada",
  fechaSolicitud: "2026-05-01T20:15:00Z",
  fechaRespuestaReceptor: "2026-05-02T08:30:00Z",
  fechaResolucion: "2026-05-02T11:40:00Z",
  resueltaPor: "user_juan_jefe",
  validacionesAutomaticas: {
    descansosOk: true,
    lineasOk: true,
    horasOk: true,
    turnosPermitidosOk: true
  }
}
```

Índices recomendados

- tenantId + centroId + estado.
- solicitanteId + estado.

- receptorId + estado.

5.18. incidencias

Propósito

Bajas, ausencias y eventos operativos que requieren modificar el cuadrante. Es el punto de entrada para la regeneración por incidencia.

Campos

Campo	Tipo	Descripción
id	string	Identificador único.
tenantId	string	Tenant.
centroId	string	Centro.
trabajadorId	string	Conductor afectado.
tipo	string	'baja_medica' 'permiso' 'falta_injustificada' 'accidente' 'otra'.
fechaInicio	string	Fecha de inicio YYYY-MM-DD.
fechaFin	string?	Fecha estimada de fin (puede ser null si indefinida).
estado	string	'pendiente_resolucion' 'resuelta' 'cerrada'.
asignacionesAfectadas	string[]	IDs de asignaciones que requieren reasignación.
registradaPor	string	Usuario que la registró.
fechaRegistro	Timestamp	Cuándo se registró.
observaciones	string?	Notas operativas.
adjuntos	string[]?	URLs a documentos adjuntos (parte médico, etc.).

Ejemplo

```
{
  id: "inc_pedro_2026_05_07",
  tenantId: "alsa",
  centroId: "alsa_murcia",
  trabajadorId: "trab_pedro_gomez",
  tipo: "baja_medica",
  fechaInicio: "2026-05-07",
  fechaFin: "2026-05-10",
  estado: "resuelta",
  asignacionesAfectadas: ["asg_pedro_2026_05_07", "asg_pedro_2026_05_08",
    "asg_pedro_2026_05_09", "asg_pedro_2026_05_10"],
  registradaPor: "user_juan_jefe",
  fechaRegistro: "2026-05-07T05:45:00Z",
  observaciones: "Parte médico recibido por WhatsApp"
}
```

Índices recomendados

- tenantId + centroId + fechaInicio.

- tenantId + trabajadorId + estado.

5.19. calendario_festivos

Propósito

Festivos aplicables a un tenant. Se precargan los nacionales y autonómicos automáticamente; el jefe de tráfico añade los locales.

Campos

Campo	Tipo	Descripción
id	string	Identificador único.
tenantId	string	Tenant.
centroId	string?	Centro (opcional, para festivos solo de un centro).
fecha	string	Fecha YYYY-MM-DD.
nombre	string	Nombre del festivo.
alcance	string	'nacional' 'autonomico' 'provincial' 'local' 'especial'.
aplicaACentros	string[]?	Si es local, a qué centros aplica (vacío = todos).
añadidoPor	string	'sistema' userId.
activo	boolean	Si está vigente.

Ejemplo

```
{
  id: "fest_alsa_2026_03_19",
  tenantId: "alsa",
  fecha: "2026-03-19",
  nombre: "San José",
  alcance: "autonomico",
  añadidoPor: "sistema",
  activo: true
}
```

Índices recomendados

- tenantId + fecha + activo.

5.20. configuracion_convenio

Propósito

Reglas legales y operativas configurables por tenant. Definen las restricciones duras que el optimizador debe respetar invariablemente. Existe un único documento por tenant.

Campos

Campo	Tipo	Descripción
id	string	Coincide con tenantId.
tenantId	string	Tenant.
descansoMinimoHoras	number	Horas mínimas entre fin de un turno y inicio del siguiente (típicamente 12).
maxHorasSemanales	number	Horas máximas semanales (típicamente 40).
maxHorasAnuales	number	Horas máximas anuales (típicamente 1.776).
minDomingosLibresAnuales	number	Mínimo de domingos libres al año.
maxFinesSemanaConsecutivosTrabajados	number	Tope de fines de semana seguidos trabajados.
maxDiasConsecutivosTrabajados	number	Días seguidos trabajados máximo (típicamente 6).
descansoSemanalMinimoHoras	number	Descanso semanal continuo mínimo (típicamente 36).
antelacionPublicacionDias	number	Días de antelación obligatoria para publicar el cuadrante.
pesoOptimizador	object	Pesos para la función objetivo: { preferenciasLibre, preferenciasTurno, equidad, ... }.
actualizadaPor	string	Super admin que la modificó.
ultimaActualizacion	Timestamp	Cuándo.

Ejemplo

```
{
  id: "conv_alsa",
  tenantId: "alsa",
  descansoMinimoHoras: 12,
  maxHorasSemanales: 40,
  maxHorasAnuales: 1776,
  minDomingosLibresAnuales: 22,
  maxFinesSemanaConsecutivosTrabajados: 2,
  maxDiasConsecutivosTrabajados: 6,
  descansoSemanalMinimoHoras: 36,
  antelacionPublicacionDias: 7,
  pesoOptimizador: {
    preferenciasLibre: 1.0,
    preferenciasTurno: 0.7,
    descansoTrasTardeMañana: 0.5,
    equidad: 0.8
  },
  actualizadaPor: "super_admin_alberto",
  ultimaActualizacion: "2026-04-15T12:00:00Z"
}
```

5.21. notificaciones

Propósito

Registro de notificaciones enviadas a usuarios. Sirve tanto para auditoría como para mostrar el centro de notificaciones dentro de la app.

Campos

Campo	Tipo	Descripción
id	string	Identificador único.
tenantId	string	Tenant.
destinatarioUserId	string	Usuario destinatario.
tipo	string	'cuadrante_publicado' 'turno_modificado' 'intercambio_solicitado' 'intercambio_aprobado' 'intercambio_rechazado' 'recordatorio_preferencias' 'asignacion_reserva' 'mensaje_jefe'.
titulo	string	Título corto.
mensaje	string	Cuerpo del mensaje.
canalesEnviados	string[]	['email', 'push', 'whatsapp'].
entidadRelacionada	object?	Referencia al objeto relacionado (asignacion, intercambio, etc.).
leida	boolean	Si el usuario la ha leído en la app.
fechaCreacion	Timestamp	Cuándo se creó.
fechaLectura	Timestamp?	Cuándo se leyó.

Ejemplo

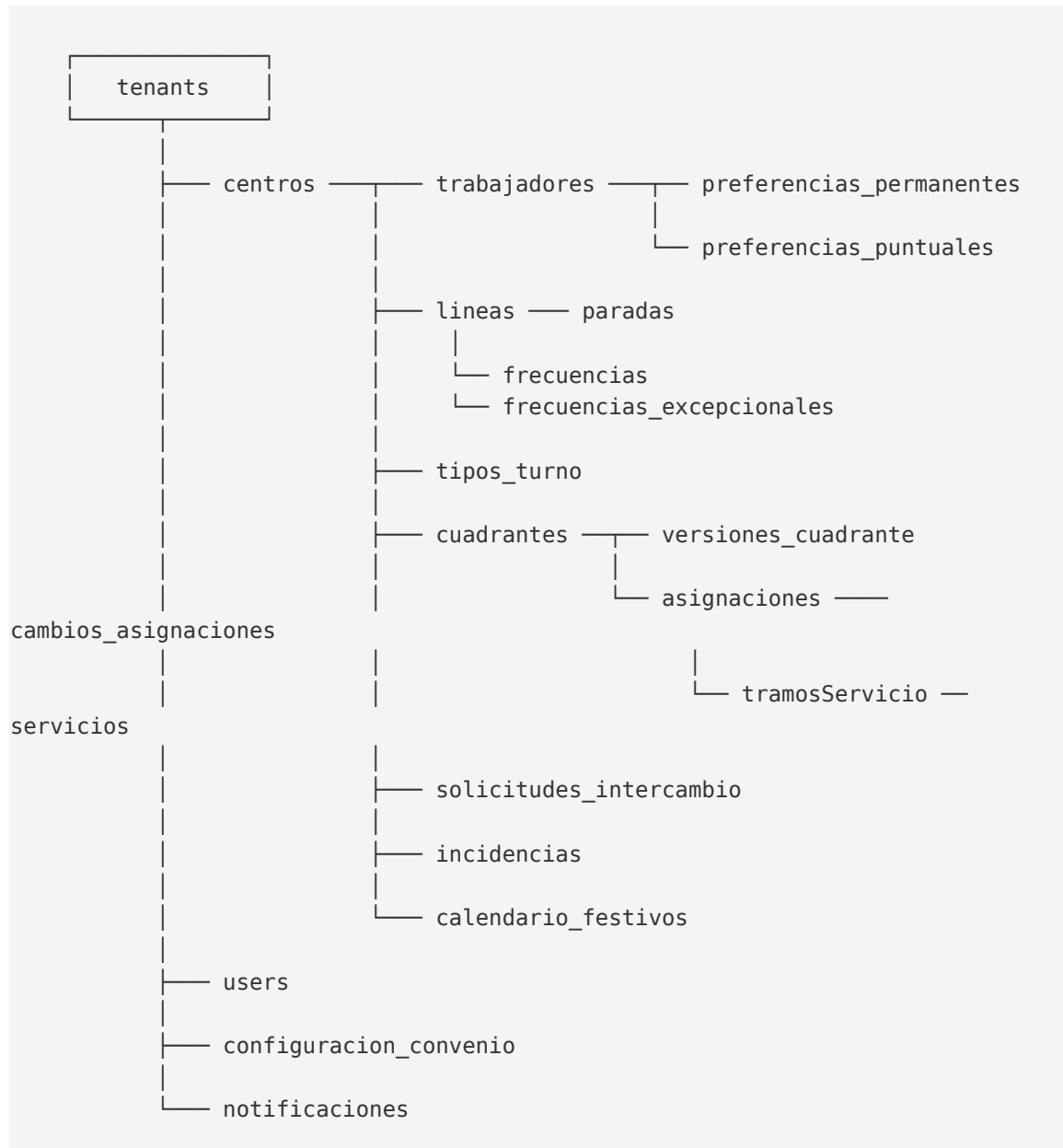
```
{
  id: "notif_pedro_2026_05_02_001",
  tenantId: "alsa",
  destinatarioUserId: "uid_firebase_auth_pedro",
  tipo: "intercambio_aprobado",
  titulo: "Tu intercambio ha sido aprobado",
  mensaje: "El jefe de tráfico ha aprobado el intercambio del 8 de mayo con Juan López.",
  canalesEnviados: ["email", "push"],
  entidadRelacionada: {
    tipo: "solicitud_intercambio",
    id: "sol_int_2026_05_02_001"
  },
  leida: false,
  fechaCreacion: "2026-05-02T11:40:00Z"
}
```

Índices recomendados

- destinatarioUserId + leida + fechaCreacion.
- tenantId + tipo + fechaCreacion.

6. Diagrama de relaciones entre entidades

El siguiente diagrama refleja las relaciones principales entre las colecciones del modelo. Los enlaces representan relaciones de referencia (un documento contiene el ID de otro).



7. Reglas de seguridad de Firestore

Las reglas de seguridad garantizan el aislamiento entre tenants y los permisos por rol. Se ejecutan en los servidores de Google antes de cada operación.

Principios

- El super_admin tiene acceso completo a todas las colecciones.
- Cualquier usuario solo puede leer y escribir documentos cuyo tenantId coincida con el suyo.
- El jefe_trafico solo puede modificar documentos de su centro.
- El conductor solo puede leer su información operativa y escribir solo en preferencias e intercambios propios.
- El tenantId, centroId y rol del usuario se obtienen del token de Firebase Auth (custom claims).

Esquema de reglas (versión simplificada)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Helpers
    function esSuperAdmin() {
      return request.auth.token.rol == 'super_admin';
    }
    function esJefeTraficoDe(tenantId, centroId) {
      return request.auth.token.rol == 'jefe_trafico'
        && request.auth.token.tenantId == tenantId
        && request.auth.token.centroId == centroId;
    }
    function esConductorDe(tenantId) {
      return request.auth.token.rol == 'conductor'
        && request.auth.token.tenantId == tenantId;
    }
    function mismoTenant(tenantId) {
      return request.auth.token.tenantId == tenantId;
    }

    // Tenants: solo super admin
    match /tenants/{tenantId} {
      allow read: if esSuperAdmin() || mismoTenant(tenantId);
      allow write: if esSuperAdmin();
    }

    // Centros: lectura mismo tenant, escritura jefe o super
    match /centros/{centroId} {
      allow read: if esSuperAdmin() || mismoTenant(resource.data.tenantId);
      allow write: if esSuperAdmin();
    }
  }
}
```

```

// Trabajadores: lectura por tenant, escritura solo jefe/super
match /trabajadores/{trabajadorId} {
  allow read: if esSuperAdmin() || mismoTenant(resource.data.tenantId);
  allow write: if esSuperAdmin()
                || esJefeTraficoDe(resource.data.tenantId,
resource.data.centroId);
}

// Asignaciones: lectura por tenant, escritura solo jefe/super
match /asignaciones/{asignacionId} {
  allow read: if esSuperAdmin() || mismoTenant(resource.data.tenantId);
  allow write: if esSuperAdmin()
                || esJefeTraficoDe(resource.data.tenantId,
resource.data.centroId);
}

// Preferencias permanentes: el conductor escribe las suyas
match /preferencias_permanentes/{prefId} {
  allow read: if esSuperAdmin()
                || esJefeTraficoDe(resource.data.tenantId,
resource.data.centroId)
                || (esConductorDe(resource.data.tenantId)
                    && resource.data.trabajadorId ==
request.auth.token.trabajadorId);
  allow write: if esSuperAdmin()
                || (esConductorDe(resource.data.tenantId)
                    && resource.data.trabajadorId ==
request.auth.token.trabajadorId);
}

// Solicitudes de intercambio: solo el solicitante puede crear, jefe
aprueba
match /solicitudes_intercambio/{solId} {
  allow read: if esSuperAdmin()
                || esJefeTraficoDe(resource.data.tenantId,
resource.data.centroId)
                || (esConductorDe(resource.data.tenantId)
                    && (resource.data.solicitanteId ==
request.auth.token.trabajadorId
                    || resource.data.receptorId ==
request.auth.token.trabajadorId));
  allow create: if esConductorDe(request.resource.data.tenantId)
                && request.resource.data.solicitanteId ==
request.auth.token.trabajadorId;
  allow update: if esJefeTraficoDe(resource.data.tenantId,
resource.data.centroId)
                || (esConductorDe(resource.data.tenantId)
                    && resource.data.receptorId ==
request.auth.token.trabajadorId);
}

// Logs de cambios: inmutables

```



```

match /cambios_asignaciones/{cambioId} {
  allow read: if esSuperAdmin() || mismoTenant(resource.data.tenantId);
  allow create: if esJefeTraficoDe(request.resource.data.tenantId,
                                request.resource.data.centroId);
  allow update, delete: if false; // immutable
}

// Configuración del convenio: solo super admin escribe
match /configuracion_convenio/{tenantId} {
  allow read: if esSuperAdmin() || mismoTenant(tenantId);
  allow write: if esSuperAdmin();
}
}

```

Notas importantes

- Las reglas presentadas son una versión simplificada. La implementación final puede incluir validaciones adicionales sobre los datos (formato, rangos, etc.).
- El log de cambios (cambios_asignaciones) se marca como immutable: solo se permite crear, no modificar ni borrar. Esto garantiza la integridad del histórico.
- Se recomienda complementar estas reglas con validaciones del lado del backend (Cloud Functions) para operaciones complejas como la generación de cuadrantes.

8. Estrategia de índices

Firestore crea automáticamente índices simples para cada campo. Los índices compuestos (sobre varios campos a la vez) deben declararse explícitamente en el archivo `firestore.indexes.json`.

Los índices clave para esta aplicación son:

Campo	Tipo	Descripción
asignaciones	tenantId + centroId + fecha (asc)	Carga del cuadrante diario.
asignaciones	tenantId + trabajadorId + fecha	Horario individual del conductor.
asignaciones	tenantId + esReserva + fecha	Listado de reservas del día.
servicios	tenantId + centroId + fecha + estado	Servicios del día y su estado.
servicios	tenantId + lineaId + fecha	Servicios de una línea concreta.
solicitudes_intercambio	tenantId + centroId + estado	Bandeja de pendientes del jefe.
solicitudes_intercambio	solicitanteId + estado	Solicitudes del conductor.
preferencias_puntuales	tenantId + fecha + estado	Carga de preferencias para optimizador.
incidencias	tenantId + centroId + fechaInicio	Incidencias del periodo.
notificaciones	destinatarioU serId + leida + fechaCreacion (desc)	Centro de notificaciones.
cambios_asignaciones	tenantId + cuadranteId + versionNueva	Historial por versión.
calendario_festivos	tenantId + fecha + activo	Festivos vigentes en una fecha.

Estos índices se declararán en el archivo `firestore.indexes.json` y se desplegarán junto con las reglas de seguridad.

9. Próximos pasos

Validado este modelo de datos, el plan de actuación es el siguiente:

- **Generación de las interfaces TypeScript:** fichero con todas las interfaces que se usarán como tipos en el backend (Cloud Functions) y en el frontend (React).
- **Setup del proyecto Firebase:** creación del proyecto, configuración de Firestore, despliegue inicial de reglas de seguridad e índices.
- **Repositorio Git:** estructura de monorepo con frontend, backend y módulo del optimizador en Python.
- **Diseño detallado del modelo matemático del optimizador:** antes de empezar a codificar el solver, formulación rigurosa de variables, restricciones y función objetivo en notación matemática.
- **Bocetos de pantallas principales:** wireframes del cuadrante, panel del conductor, mercado de intercambios y dashboard de equidad.
- **Inicio del desarrollo iterativo:** primeras semanas centradas en autenticación, multi-tenant y CRUDs básicos.

Fin del documento

Modelo de datos v1.0 — Mayo 2026